

Smart Blind Assistant

Architecture simplifiée — Projet Fin de Module

Version épurée pour un projet de fin de module (3ème année IA & Data Science). Chaque dossier a **un seul rôle clair**. Pas de surcharge.

PARTIE 1 — Backend Python (FastAPI)

Structure du dossier

Seulement **5 fichiers principaux** + 1 dossier par module IA. C'est tout.

```
backend/
├── main.py           ← point d'entrée, lance le serveur FastAPI
├── requirements.txt  ← liste des librairies à installer
├── .env              ← variables de config (port, chemins modèles...)
├── cv/
│   ├── detector.py   ← détection obstacles avec YOLO
│   ├── finder.py     ← trouver un objet dans l'image
│   └── localizer.py  ← identifier la pièce courante
├── nlp/
│   ├── intent.py     ← comprendre la commande (FIND / NAVIGATE / SOS)
│   └── intents.json  ← liste des intentions + exemples de phrases
├── navigation/
│   ├── graph.py      ← maison modélisée en graphe (salon→couloir→cuisine)
│   ├── pathfinder.py ← calcule le chemin optimal entre 2 pièces (Dijkstra)
│   ├── instructions.py ← convertit le chemin en consignes vocales
│   │   ex: ["avancez", "tournez à droite", "vous êtes arrivé"]
│   └── house.json    ← config des pièces et connexions
└── safety/
    └── sos.py        ← gérer l'urgence (appel / message)
```

Ce que fait chaque fichier

Fichier	Rôle en une ligne
main.py	Déclare les routes API (/detect, /navigate, /sos...)
cv/detector.py	Prend une image → retourne les obstacles détectés
nlp/intent.py	Prend du texte → retourne FIND_OBJECT / NAVIGATE / SOS
navigation/graph.py	Modélise la maison : pièces + connexions entre elles
navigation/pathfinder.py	Calcule le chemin le plus court entre 2 pièces (Dijkstra)
navigation/instructions.py	Traduit le chemin en consignes vocales (avancez, tournez...)
safety/sos.py	Envoie un SMS / appel d'urgence

■ Conseil : commence par main.py + cv/detector.py. Le reste vient après.

PARTIE 2 — App Mobile (Flutter)

Structure du dossier

Seulement **4 écrans** + 1 dossier services pour parler au backend. Simple et direct.

```
flutter_app/
├──
├── lib/
│   ├──
│   ├── main.dart          ← point d'entrée de l'app
│   ├──
│   ├── screens/          ← les 4 écrans de l'app
│   │   ├── home_screen.dart ← écran principal (grand bouton micro)
│   │   ├── navigate_screen.dart ← guidage pas à pas
│   │   ├── sos_screen.dart  ← écran urgence
│   │   └── settings_screen.dart ← vitesse voix, contacts...
│   ├──
│   ├── services/         ← communication avec le backend
│   │   ├── api_service.dart ← appels HTTP vers FastAPI
│   │   ├── tts_service.dart ← Text-to-Speech (lire les réponses)
│   │   └── stt_service.dart ← Speech-to-Text (écouter l'utilisateur)
│   ├──
│   ├── widgets/          ← composants réutilisables
│   │   ├── mic_button.dart ← grand bouton micro central
│   │   └── danger_alert.dart ← alerte rouge si obstacle proche
│   ├──
│   ├── assets/
│   │   ├── sounds/
│   │   │   └── alert.mp3 ← son d'alerte danger
│   │   └──
│   └── pubspec.yaml      ← dépendances Flutter
```

Librairies Flutter à installer

Besoin	Package Flutter
Appels API backend	http
Text-to-Speech	flutter_tts
Speech-to-Text	speech_to_text
Caméra	camera
Vibration	vibration
GPS (SOS)	geolocator

■ Conseil : commence par `home_screen.dart` + `api_service.dart`. C'est le cœur de l'app.

Ordre conseillé pour développer

Étape	Quoi faire	Fichiers concernés
1	Monter le serveur FastAPI vide avec toutes les routes	main.py
2	Brancher YOLO pour la détection d'obstacles	cv/detector.py
3	Faire l'intent detection NLP (simple cosin)	nlp/intent.py
4	Coder le graphe maison + navigation	navigation/graph.py
5	Créer l'écran principal Flutter + bouton micro	home_screen.dart
6	Brancher STT → API → TTS dans l'app	stt/tts/api_service.dart

7	Ajouter l'écran navigation + SOS	navigate_screen / sos_screen
---	----------------------------------	------------------------------